МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Лабораторна робота №3

з курсу: «Технології побудови розподілених баз даних та знань»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Практичні завдання

1. З тестовими даним, відмінними від вказаних, виконуємо ознайомчу частину.

2. Зчитаємо кількість документів по сегментам. Запитом PUT оновимо екземпляр даних. Утилітою cURL зчитаємо ідентичний документ з вузла, щодо котрого виконано реплікацію (дублювання).

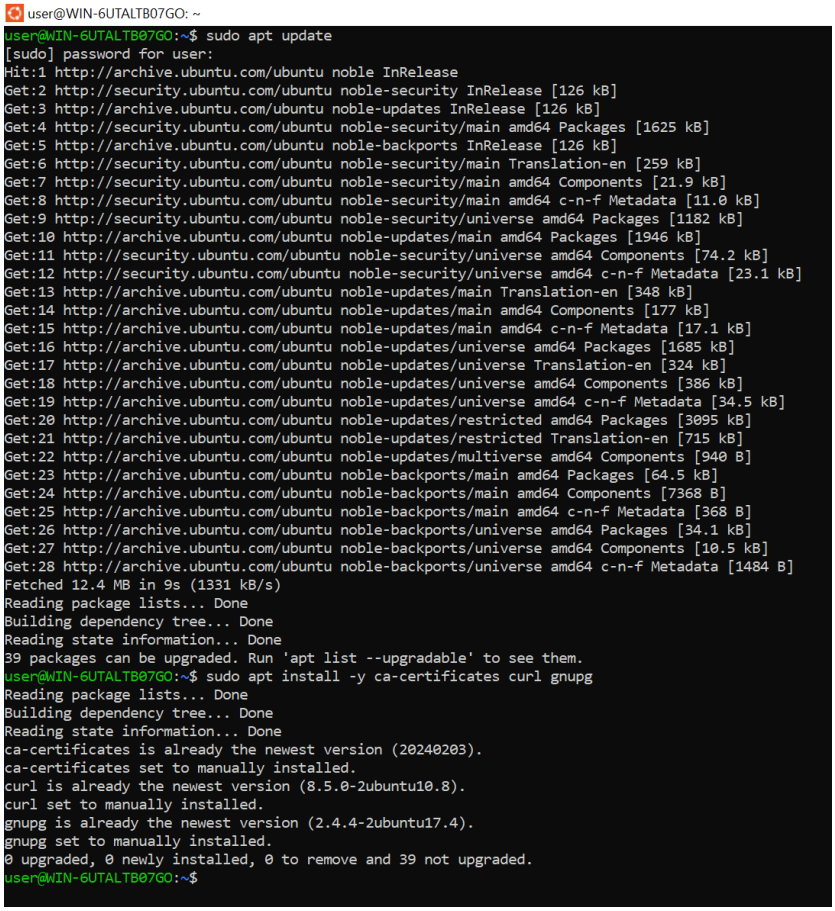
3. Утилітою docker-compose й опцією up включимо додатково два керівних вузла й два вузла даних. Дозапишемо POST-запитом 5 документів. Побудуємо індекс і зчитаємо екземпляри по вмісту полів.

4. Утилітою docker-compose й опцією down вимкнемо дійсний основний вузол і опцією up включимо новий основний вузол. Зчитає-мо сегменти кластера _cat/shards.

Хід роботи

1. Оновлення та базові пакети

Для початку виконаємо онвлення, а саме оновимо списки пакетів, встановлено інструменти для безпечного завантаження репозиторіїв і роботи з HTTPS.



2. Docker

Встановлено Docker Engine та плагін Docker Compose v2 (docker compose). Це дозволяє ізоловано розгорнути кластер Elasticsearc**.**

```
user@WIN-6UTALTB07GO:~$ sudo install -m 0755 -d /etc/apt/keyrings
user@WIN-6UTALTB07GO:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
user@WIN-6UTALTB07GO:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
user@WIN-6UTALTB07GO:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker
/etc/apt/sources.list.d/docker.list > /dev/null
user@WIN-6UTALTB07GO:~$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:2 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [52.9 kB]
Hit:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 101 kB in 1s (172 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
user@WIN-6UTALTB07GO:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Виконаємо перевірку

```
user@WIN-6UTALTB07GO:~$ sudo docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

user@WIN-6UTALTB07GO:~$ docker compose version
Docker Compose version v5.1.3
user@WIN-6UTALTB07GO:~$
```

```
user@WIN-6UTALTB07GO:~$ docker compose version
Docker Compose version v5.1.3
user@WIN-6UTALTB07GO:~$ sudo usermod -aG docker "$USER"
user@WIN-6UTALTB07GO:~$ sudo sysctl -w vm.max_map_count=524288
vm.max_map_count = 524288
user@WIN-6UTALTB07GO:~$ echo "vm.max_map_count=524288" | sudo tee /etc/sysctl.d/99-elasticsearch.conf
vm.max_map_count=524288
user@WIN-6UTALTB07GO:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-zeropage.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/99-elasticsearch.conf ...
* Applying /usr/lib/sysctl.d/99-protect-links.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.conf ...
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
kernel.pid_max = 4194304
vm.max_map_count = 524288
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
```

3. Підготовка docker-compose.yml і запуск кластера

Створюємо окремий каталог для конфігурації лабораторної, щоб усі шляхи та файли були в одному місці.

```
user@WIN-6UTALTB07GO:~$ mkdir -p ~/elasticsearch-lab
user@WIN-6UTALTB07GO:~$ cd ~/elasticsearch-lab
```

Щоб переконатися, що ядро Linux дозволяє Elasticsearch коректну роботу з mmap перевіримо його значення. Якщо б значення було меншим, потрібно було б виконати sudo sysctl -w vm.max_map_count=524288 і зафіксувати в /etc/sysctl.d/… для постійної дії після перезавантаження.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ sysctl vm.max_map_count
vm.max_map_count = 524288
```

Створюємо yml файл з описом чотирьох сервісів: три вузли Elasticsearch (es0, es1, es2) та Kibana для веб-інтерфейсу

user@WIN-6UTALTB07GO: ~/elasticsearch-lab

GNU nano 7.2

```yaml
version: '2.4'

services:
  es0:
    image: docker.elastic.co/elasticsearch/elasticsearch:6.8.6
    container_name: es0
    environment:
      - ES_SKIP_SET_KERNEL_PARAMETERS=true
      - cluster.name=docker-cluster
      - node.name=es0
      - bootstrap.memory_lock=true
      - "ES_JAVA_OPTS=-Xms256m -Xmx256m"
      - "discovery.zen.ping.unicast.hosts=es0,es1,es2"
      - "discovery.zen.minimum_master_nodes=2"
      - "xpack.security.enabled=false"
      - "xpack.watcher.enabled=false"
      - "xpack.monitoring.enabled=false"
    ulimits:
      memlock:
        soft: -1
        hard: -1
    mem_limit: 1g
    volumes:
      - esvol0:/usr/share/elasticsearch/data
    ports:
      - "9200:9200"
    networks:
      - esnet

  es1:
    image: docker.elastic.co/elasticsearch/elasticsearch:6.8.6
    container_name: es1
    environment:
      - ES_SKIP_SET_KERNEL_PARAMETERS=true
      - cluster.name=docker-cluster
      - node.name=es1
      - bootstrap.memory_lock=true
      - "ES_JAVA_OPTS=-Xms256m -Xmx256m"
      - "discovery.zen.ping.unicast.hosts=es0,es1,es2"
      - "discovery.zen.minimum_master_nodes=2"
      - "xpack.security.enabled=false"
      - "xpack.watcher.enabled=false"
      - "xpack.monitoring.enabled=false"
    depends_on:
      - es0
    ulimits:
      memlock:
        soft: -1
        hard: -1
    mem_limit: 1g
    volumes:
      - esvol1:/usr/share/elasticsearch/data
```

cluster.name — одна назва, щоб усі три контейнери утворили один кластер.

node.name — унікальне ім’я вузла (видно в _cat/nodes).

discovery.zen.ping.unicast.hosts — список хостів для взаємного «пошуку» вузлів у мережі Compose (es0, es1, es2 резолвяться іменами сервісів).

discovery.zen.minimum_master_nodes=2 — мінімальна кількість майстер-груп для обрання master при трьох вузлах (типова налаштування під кворум для 3 master-eligible у старому Zen).

ES_JAVA_OPTS=-Xms256m -Xmx256m — обмеження heap JVM (важливо для слабких ПК / WSL).

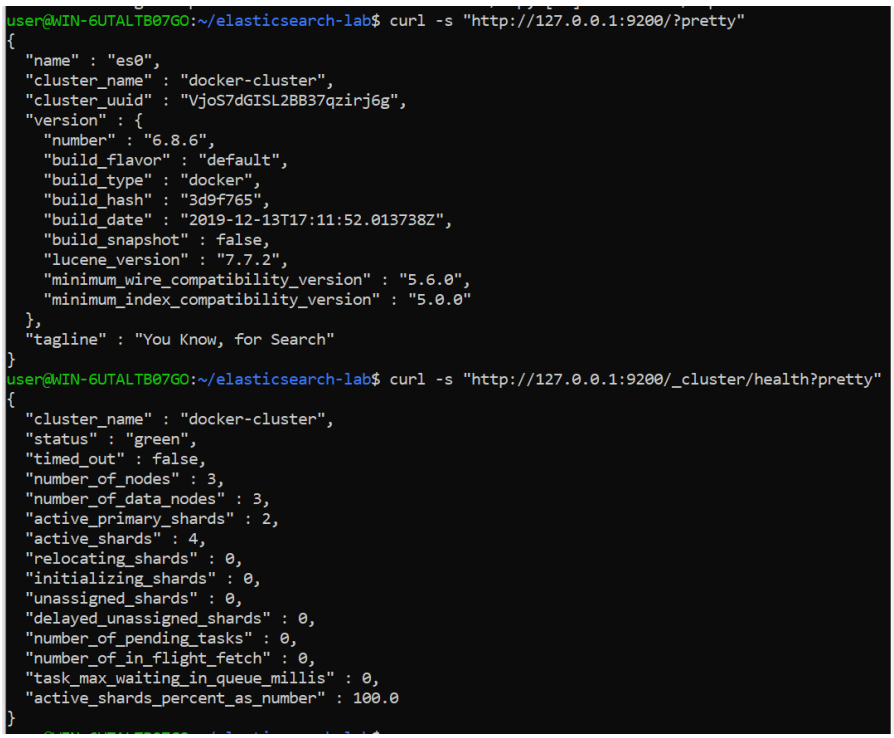
ports: "9200:9200" лише на es0 — з вашої Ubuntu/WSL до REST API кластеру звертаємось на localhost:9200; інші вузли доступні через внутрішню мережу Docker.

mem_limit / ulimits memlock — наближено відповідають вимогам Elasticsearch до пам’яті та блокування; без цього можливі попередження або збої.

Запустимо та перевіримо наші контейнери.



Виконаємо перевірку API Elasticsearch



```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/nodes?v"
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
172.19.0.2           59          43   1    0.00    0.07     0.09 mdi       -      es0
172.19.0.3           57          43   0    0.00    0.07     0.09 mdi       -      es2
172.19.0.5           51          43   0    0.00    0.07     0.09 mdi       *      es1
```

Тобто ми підтвердили, що кластер не лише запущений, а й узгоджений: вузли бачать одне одного, обраний master, здоров’я кластеру відображається API.

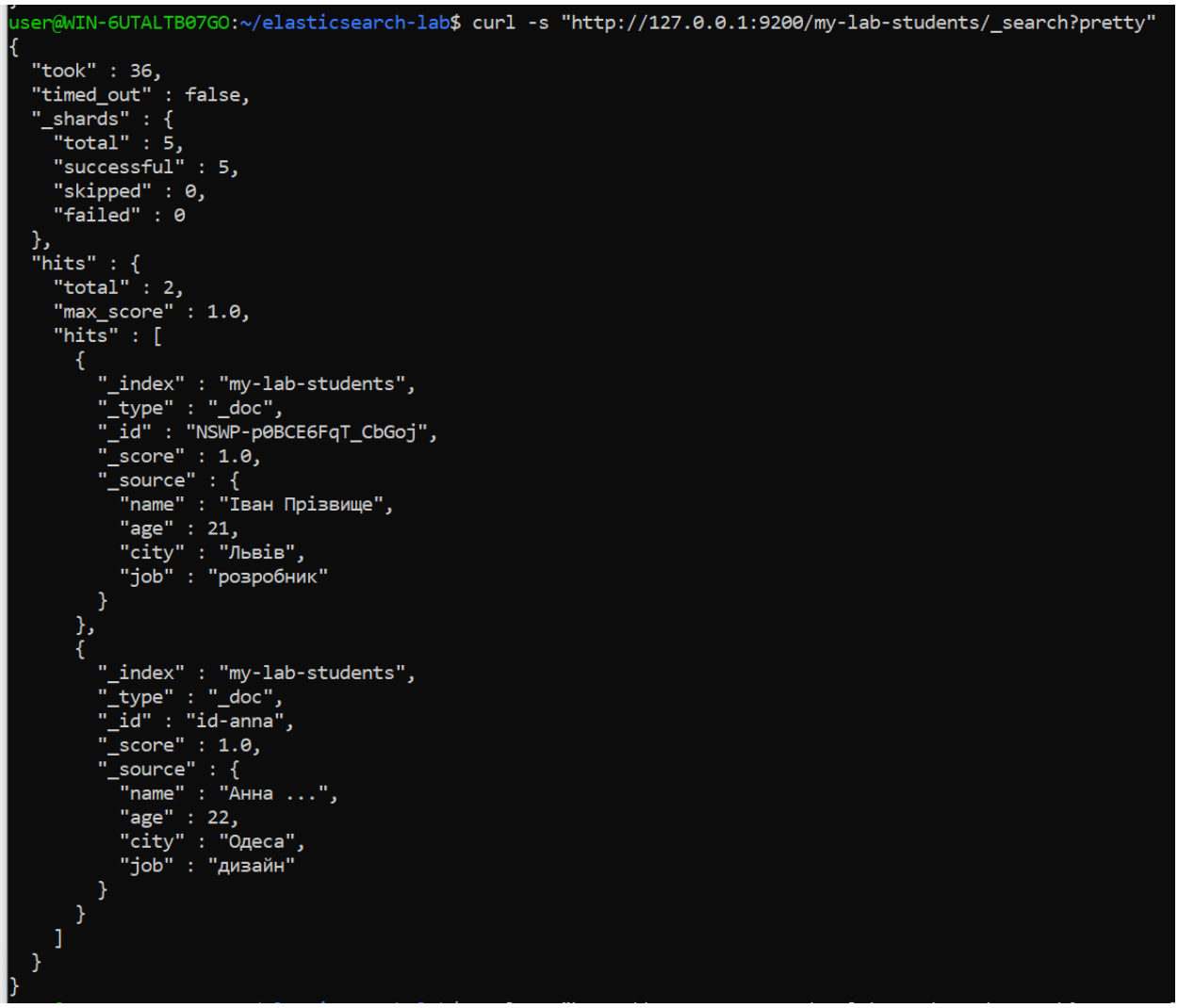
4. Ознайомча частина зі своїми даними

Виконаємо ознайомчу частину з власним індексом my-lab-students і даними, відмінними від навчального прикладу: перевірено стан кластера та вузлів, створено індекс, проаналізовано primary/replica шарди, виконано індексацію методами POST та PUT, виконано повнотекстовий та фільтраційний пошук методом URI-параметра q, а також запитами Query DSL типів match, wildcard та range.

У розподіленому Elasticsearch узгодженість і доступність індексів відображаються узагальненими показниками здоров’я кластера. Перевірка /_cluster/health показує кількість вузлів, роль data-вузлів та стан шардів узагальнено; статус «green» означає, що всі призначені primary і replica доступні.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cluster/health?pretty"
{
  "cluster_name" : "docker-cluster",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 3,
  "number_of_data_nodes" : 3,
  "active_primary_shards" : 7,
  "active_shards" : 14,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
```

Найпростіший пошук без фільтра повертає всі документи індексу (узгоджено обходяться primary shard’и індексу). Це базова перевірка після індексації: видно _id, _source і кількість hits.total, щоб підтвердити наявність обох записів у тестовому індексі та коректність збережених полів.



Простий синтаксис запиту в рядку URL зручний для швидких перевірок: умова подається у полі q, наприклад точна відповідність числового поля. Elasticsearch інтерпретує це як запит до повнотекстового/полевого пошуку залежно від маппінгу поля. Це потрібно щоб відпрацювати типовий навчальний сценарій методички (пошук параметром у рядку) і отримати підмножину документів

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/my-lab-students/_search?q=age:21&pretty"

{
  "took" : 198,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 1,
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "my-lab-students",
        "_type" : "_doc",
        "_id" : "NSWP-p0BCE6FqT_CbGoj",
        "_score" : 1.0,
        "_source" : {
          "name" : "Іван Прізвище",
          "age" : 21,
          "city" : "Львів",
          "job" : "розробник"
        }
      }
    ]
  }
}
```

Запити з тілом JSON дозволяють використовувати Query DSL; оператор match виконує аналіз вхідного рядка і пошук з урахуванням токенізації тексту, що зручно для текстових полів. Покажемо пошук за змістом текстового поля «місто» і перевірити, що повертається

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9
200/my-lab-students/_search?pretty" -d '{"query":{"match":{"city":"Одеса"}}}'
{
  "took" : 242,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 1,
    "max_score" : 0.6931472,
    "hits" : [
      {
        "_index" : "my-lab-students",
        "_type" : "_doc",
        "_id" : "id-anna",
        "_score" : 0.6931472,
        "_source" : {
          "name" : "Анна ...",
          "age" : 22,
          "city" : "Одеса",
          "job" : "дизайн"
        }
      }
    ]
  }
}
```

Запит через оператор wildcard дозволяє задавати шаблон з символами * та ? для рядкових полів і відбирає документи за частковою відповідністю. Виконаємо цей запит щоб ілюструвати пошук за префіксом значення професії (знайдено запис «розробник» відповідно до шаблону «роз*»).

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9
200/my-lab-students/_search?pretty" -d '{"query":{"wildcard":{"job":"роз*"}}}'
{
  "took" : 60,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 1,
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "my-lab-students",
        "_type" : "_doc",
        "_id" : "NSWP-p0BCE6FqT_CbGoj",
        "_score" : 1.0,
        "_source" : {
          "name" : "Іван Прізвище",
          "age" : 21,
          "city" : "Львів",
          "job" : "розробник"
        }
      }
    ]
  }
}
```

Умова range застосовується до упорядкованих полів (типово числових або дат) і вибирає документи, чиє значення потрапляє в закритий чи напівзакритий інтервал

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9
200/my-lab-students/_search?pretty" -d '{"query":{"range":{"age":{"gte":20,"lte":22}}}}'
{
  "took" : 19,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 2,
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "my-lab-students",
        "_type" : "_doc",
        "_id" : "NSWP-p0BCE6FqT_CbGoj",
        "_score" : 1.0,
        "_source" : {
          "name" : "Іван Прізвище",
          "age" : 21,
          "city" : "Львів",
          "job" : "розробник"
        }
      },
      {
        "_index" : "my-lab-students",
        "_type" : "_doc",
        "_id" : "id-anna",
        "_score" : 1.0,
        "_source" : {
          "name" : "Анна ...",
          "age" : 22,
          "city" : "Одеса",
          "job" : "дизайн"
        }
      }
    ]
  }
}
```

5. Зчитаємо кількість документів по сегментам. Запитом PUT оновимо екземпляр даних. Утилітою cURL зчитаємо ідентичний документ з вузла, щодо котрого виконано реплікацію (дублювання).

Розподіл даних індексу в Elasticsearch задається множиною primary шардів і їх реплік. Кількість документів на кожній копії шарду відображається в уніфікованому API _cat/shards: для кожного номера шарду є рядок p (primary) та r (replica); колонка docs показує накопичені на цій копії документи. Загальна кількість у індексі перевіряється запитом _count.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/my-lab-students/_count?pretty"
{
  "count" : 2,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  }
}
```

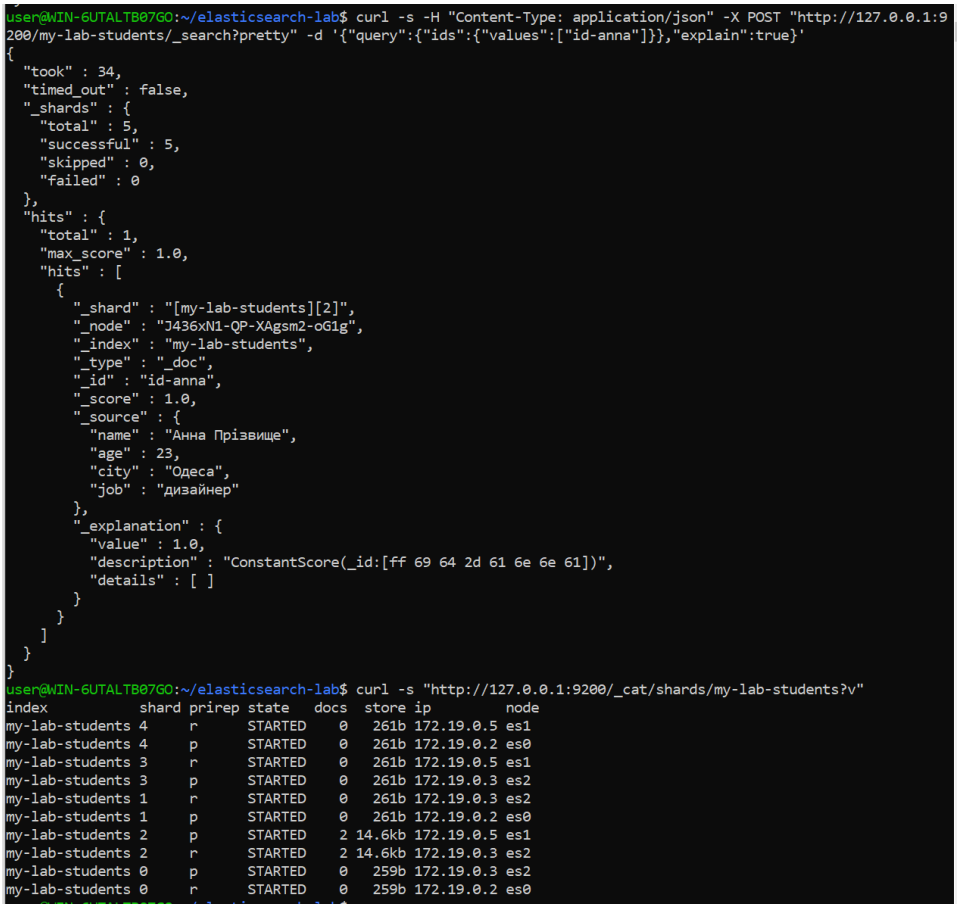
```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/shards/my-lab-students?v"
index           shard prirep state   docs store ip         node
my-lab-students 4     r      STARTED    0  261b 172.19.0.5 es1
my-lab-students 4     p      STARTED    0  261b 172.19.0.2 es0
my-lab-students 3     r      STARTED    0  261b 172.19.0.5 es1
my-lab-students 3     p      STARTED    0  261b 172.19.0.3 es2
my-lab-students 1     r      STARTED    0  261b 172.19.0.3 es2
my-lab-students 1     p      STARTED    0  261b 172.19.0.2 es0
my-lab-students 2     p      STARTED    2 9.8kb 172.19.0.5 es1
my-lab-students 2     r      STARTED    2 9.8kb 172.19.0.3 es2
my-lab-students 0     p      STARTED    0  259b 172.19.0.3 es2
my-lab-students 0     r      STARTED    0  259b 172.19.0.2 es0
```

Бачимо, що сума docs по всіх рядках p дорівнює логічній кількості документів індексу (кожен документ живе на одному primary), репліки дублюють ті самі дані (у стабільному кластері docs на r узгоджуються з відповідним p).

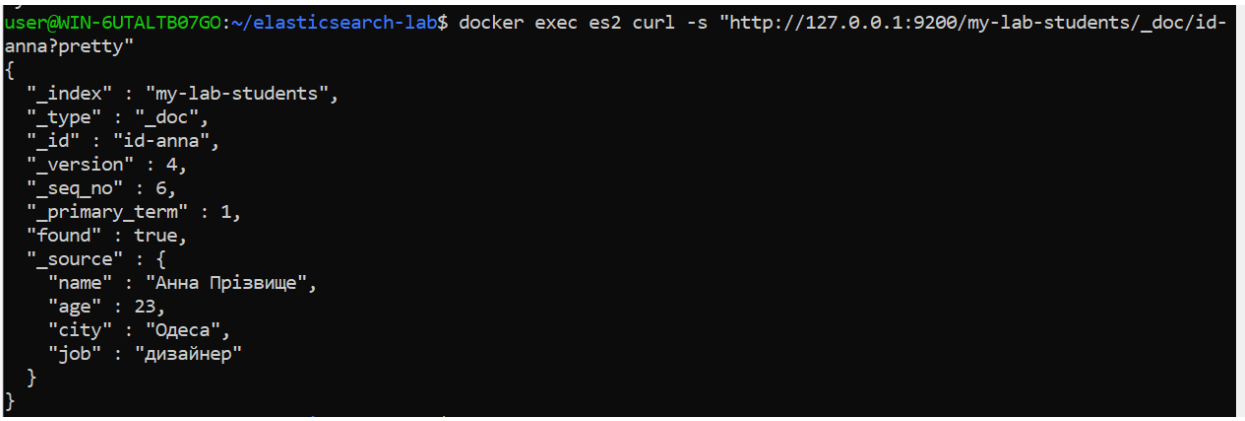
Оновлення існуючого екземпляра з явним _id у Elasticsearch 6.x здійнюється методом PUT на шлях /{індекс}/_doc/{id}: передано нове JSON-тіло, збільшується _version, змінюються записи primary і синхронізуються репліки.



Ідентифікатор документа під час індексації визначає маршрутизацію до одного из primary шардів; replica копія розміщується на іншому data-узлі. За таблицею _cat/shards визначаємо номер шарду та назву вузла (node), де розміщена репліка r для цього шарду.



Для підтвердження наявності дубльованої копії виконано HTTP GET до API Elasticsearch на вузлі, який згідно з _cat/shards зберігає replica відповідного шарду. Використано docker exec, оскільки порти 9200 зовні проброшені лише на es0, тоді як es1/es2 доступні всередині Docker-мережі.



6. Утилітою docker-compose й опцією up включимо додатково два керівних вузла й два вузла даних. Дозапишемо POST-запитом 5 документів. Побудуємо індекс і зчитаємо екземпляри по вмісту полів.

Далі виконаємо масштабуванню кластера із розділенням ролей: частина вузлів налаштована як кандидати в майстри без зберігання даних користувацьких шардів, частина - як вузли даних без голосування за обрання master згідно з моделлю виробничого профілю розгортання.

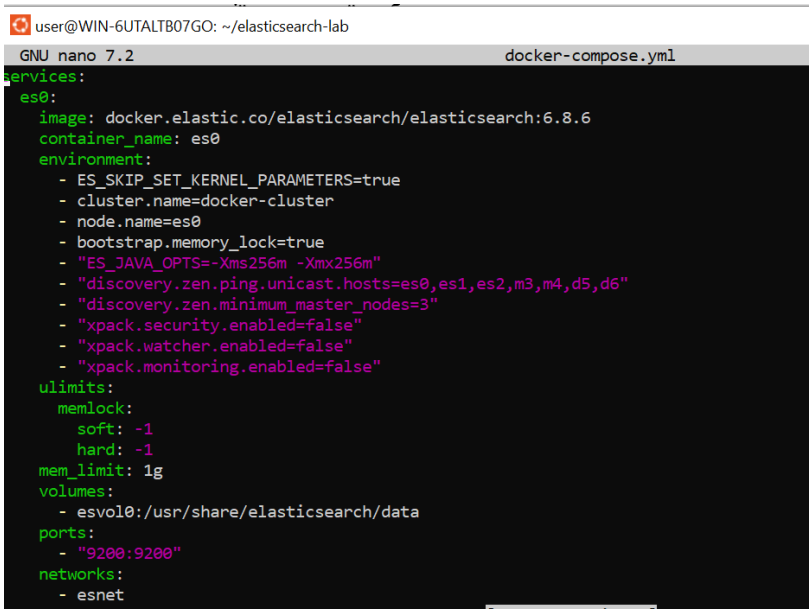
Для узгодженого обрання керівника в Elasticsearch 6.x при розширенні набору master-eligible необхідно оновити перелічення вузлів у discovery.zen.ping.unicast.hosts та змінити discovery.zen.minimum_ master_nodes відповідно до правила кворуму. Після стабілізації кластеру створюється новий індекс для окремої серії експериментів, індексується п’ять документів та виконуються пошукові запити за значеннями полів для демонстрації коректної роботи повнотекстового та структурного пошуку на розширеній топології.

У всіх семи Elasticsearch-сервісах мають збігатися cluster.name, один і той самий список discovery.zen.ping.unicast.hosts і minimum_master_nodes=3.

Вузли es0, es1, es2 лишаються ті самі master+data. Додаються:

- m3, m4 — node.master=true, node.data=false
- d5, d6 — node.master=false, node.data=true

При п’яти майстер-допустимих вузлах (es0, es1, es2, m3, m4) кворум: $\lfloor N/2 \rfloor + 1 = 3$. Вузли d5 і d6 у майстер-голосування не входять.



Виконаємо перезапуск стеку.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ docker compose down
[+] down 5/5
 ✔ Container es1                    Removed                                                                    1.0s
 ✔ Container kibana                 Removed                                                                    1.5s
 ✔ Container es2                    Removed                                                                    0.9s
 ✔ Container es0                    Removed                                                                   10.5s
 ✔ Network elasticsearch-lab_esnet  Removed                                                                    0.3s
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ docker compose up -d
[+] up 13/13
 ✔ Network elasticsearch-lab_esnet  Created                                                                    0.0s
 ✔ Volume elasticsearch-lab_esvold5 Created                                                                    0.0s
 ✔ Volume elasticsearch-lab_esvold6 Created                                                                    0.0s
 ✔ Volume elasticsearch-lab_esvolm3 Created                                                                    0.0s
 ✔ Volume elasticsearch-lab_esvolm4 Created                                                                    0.0s
 ✔ Container es0                    Started                                                                    0.7s
 ✔ Container es1                    Started                                                                    1.0s
 ✔ Container m3                     Started                                                                    0.9s
 ✔ Container m4                     Started                                                                    1.0s
 ✔ Container d6                     Started                                                                    1.1s
 ✔ Container d5                     Started                                                                    1.1s
 ✔ Container kibana                 Started                                                                    1.1s
 ✔ Container es2                    Started                                                                    0.9s
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ docker compose ps
NAME      IMAGE                                                  COMMAND                  SERVICE   CREATED
   STATUS          PORTS
d5        docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   d5        12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
d6        docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   d6        12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
es0       docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   es0       12 seconds ago
   Up 11 seconds   0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 9300/tcp
es1       docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   es1       12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
es2       docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   es2       12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
kibana    docker.elastic.co/kibana/kibana:6.8.6                  "/usr/local/bin/kiba…"   kibana    12 seconds ago
   Up 11 seconds   0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
m3        docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   m3        12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
m4        docker.elastic.co/elasticsearch/elasticsearch:6.8.6    "/usr/local/bin/dock…"   m4        12 seconds ago
   Up 11 seconds   9200/tcp, 9300/tcp
```

Перевірка

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cluster/health?pretty"
{
  "cluster_name" : "docker-cluster",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 7,
  "number_of_data_nodes" : 5,
  "active_primary_shards" : 7,
  "active_shards" : 14,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/nodes?v"
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
172.19.0.4           43          78   4    0.45    1.01     0.49 mi        -      m3
172.19.0.3           48          78   4    0.45    1.01     0.49 mdi       -      es2
172.19.0.5           50          78   4    0.45    1.01     0.49 mdi       *      es1
172.19.0.6           56          78   4    0.45    1.01     0.49 mi        -      m4
172.19.0.9           53          78   4    0.45    1.01     0.49 di        -      d5
172.19.0.2           61          78   4    0.45    1.01     0.49 mdi       -      es0
172.19.0.8           48          78   4    0.45    1.01     0.49 di        -      d6
```

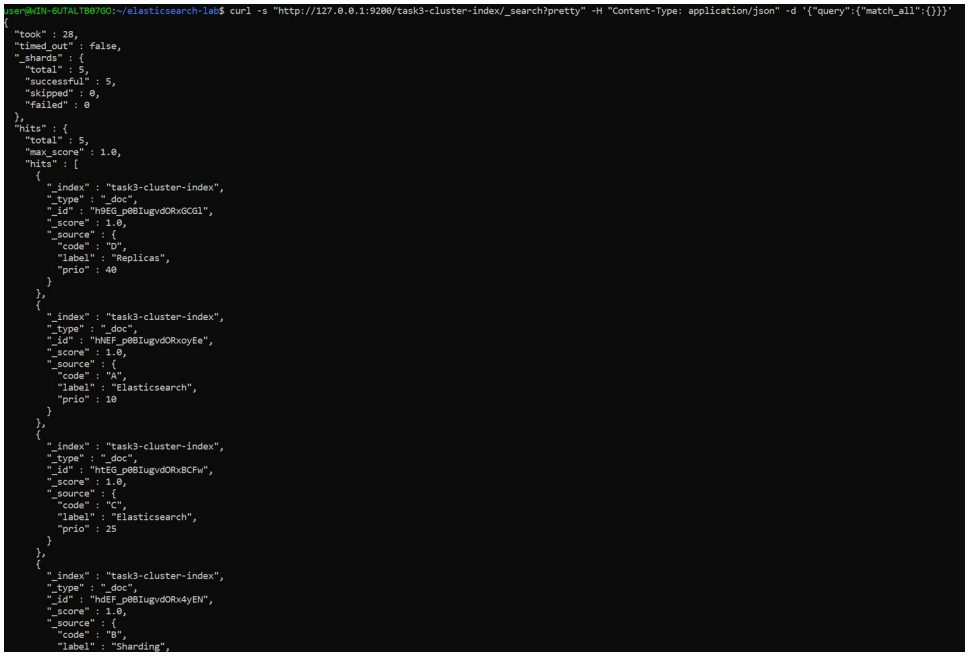
Після заміни файлу конфігурації Docker Compose виконано перезапуск стеку; усі семи служб Elasticsearch активні. Перевіркою _cluster/health підтверджено сім узлів із п’ятьма вузлами даних та стан кластера green; таблиця _cat/nodes продемонструвала наявність окремих вузлів майстробезданих із ролями типу mi та виключних вузлів даних із ролями di, а також активний майстербузол із позначенням.

Новий індекс + 5 документів POST + пошук по полях

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9
200/task3-cluster-index/_doc/" \
>   -d '{"code":"B","label":"Sharding","prio":30}'
{"_index":"task3-cluster-index","_type":"_doc","_id":"hdEF_p0BIugvdORx4yEN","_version":1,"result":"created","_sha
rds":{"total":2,"successful":2,"failed":0},"_seq_no":0,"_primary_term":1}user@WIN-6UTALTB07GO:~/elasticsearch-lab
$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9200/task3-cluster-index/_doc/" \
-d '{>   -d '{"code":"C","label":"Elasticsearch","prio":25}'
{"_index":"task3-cluster-index","_type":"_doc","_id":"htEG_p0BIugvdORxBCFw","_version":1,"result":"created","_sha
rds":{"total":2,"successful":2,"failed":0},"_seq_no":1,"_primary_term":1}user@WIN-6UTALTB07GO:~/elasticsearch-lab
$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9200/task3-cluster-index/_doc/" \
>   -d '{"code":"D","label":"Replicas","prio":40}'
{"_index":"task3-cluster-index","_type":"_doc","_id":"h9EG_p0BIugvdORxGCGl","_version":1,"result":"created","_sha
rds":{"total":2,"successful":2,"failed":0},"_seq_no":0,"_primary_term":1}user@WIN-6UTALTB07GO:~/elasticsearch-lab
$ curl -s -H "Content-Type: application/json" -X POST "http://127.0.0.1:9200/task3-cluster-index/_doc/" \
-d '{>   -d '{"code":"E","label":"Zen discovery","prio":15}'
{"_index":"task3-cluster-index","_type":"_doc","_id":"iNEG_p0BIugvdORxLiEr","_version":1,"result":"created","_sha
rds":{"total":2,"successful":2,"failed":0},"_seq_no":1,"_primary_term":1}user@WIN-6UTALTB07GO:~/elasticsearch-lab
```

Перевірка

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/task3-cluster-index/_count?pretty"
{
  "count" : 5,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  }
}
```



```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/task3-cluster-index/_search?pretty" -H "Content-Type: application/json" -d '{"query":{"match_all":{}}}'
{
  "took" : 28,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 5,
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "h9EG_p0BIugvdORxGCGl",
        "_score" : 1.0,
        "_source" : {
          "code" : "D",
          "label" : "Replicas",
          "prio" : 40
        }
      },
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "hNEF_p0BIugvdORxoyEe",
        "_score" : 1.0,
        "_source" : {
          "code" : "A",
          "label" : "Elasticsearch",
          "prio" : 10
        }
      },
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "htEG_p0BIugvdORxBCFw",
        "_score" : 1.0,
        "_source" : {
          "code" : "C",
          "label" : "Elasticsearch",
          "prio" : 25
        }
      },
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "hdEF_p0BIugvdORx4yEN",
        "_score" : 1.0,
        "_source" : {
          "code" : "B",
          "label" : "Sharding",
```

Пошук по полях:

```
rl -s -H "Content-Type: application/json" \
>   -X POST "http://127.0.0.1:9200/task3-cluster-index/_search?pretty" \
earch">   -d '{"query":{"match":{"label":"Elasticsearch"}}}'
{
  "took" : 26,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : 2,
    "max_score" : 0.18232156,
    "hits" : [
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "hNEF_p0BIugvdORxoyEe",
        "_score" : 0.18232156,
        "_source" : {
          "code" : "A",
          "label" : "Elasticsearch",
          "prio" : 10
        }
      },
      {
        "_index" : "task3-cluster-index",
        "_type" : "_doc",
        "_id" : "htEG_p0BIugvdORxBCFw",
        "_score" : 0.18232156,
        "_source" : {
          "code" : "C",
          "label" : "Elasticsearch",
          "prio" : 25
        }
      }
    ]
  }
}
```

7. Утилітою docker-compose й опцією down вимкнемо дійсний основний вузол і опцією up включимо основний вузол. Зчитаємо сегменти кластера _cat/shards.

Для виконання завдання робимо наступне:

1. docker compose stop es1 — зупинено вузол es1, який перед цим був master (*).

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/nodes?v"
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
172.19.0.9           56          74   0    0.09    0.08     0.14 di        -      d5
172.19.0.4           51          74   0    0.09    0.08     0.14 mi        -      m3
172.19.0.8           56          74   0    0.09    0.08     0.14 di        -      d6
172.19.0.5           44          74   0    0.09    0.08     0.14 mdi       *      es1
172.19.0.6           46          74   0    0.09    0.08     0.14 mi        -      m4
172.19.0.3           51          74   0    0.09    0.08     0.14 mdi       -      es2
172.19.0.2           51          74   0    0.09    0.08     0.14 mdi       -      es0
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ docker compose stop es1
[+] stop 1/1
 ⠿ Container es1 Stopped
```

2. Кластер лишився життєздатним:

number_of_nodes": 6 (було 7 - один вузол вимкнено).

number_of_data_nodes": 4 (бо es1 був mdi - і master, і data).

status": "green", узгодженість шардів відновлена, нез призначені шарди 0.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cluster/health?pretty"
{
  "cluster_name" : "docker-cluster",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 6,
  "number_of_data_nodes" : 4,
  "active_primary_shards" : 12,
  "active_shards" : 24,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
```

3. Перевибор master: тепер * на m4 — кластер автоматично обрав нового майстербузол серед master-eligible (у тебе es0, es2, m3, m4).

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/nodes?v"
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
172.19.0.8           49          67   0    0.08    0.11     0.13 di        -      d6
172.19.0.9           51          67   0    0.08    0.11     0.13 di        -      d5
172.19.0.2           61          67   0    0.08    0.11     0.13 mdi       -      es0
172.19.0.6           45          67   0    0.08    0.11     0.13 mi        *      m4
172.19.0.4           63          67   0    0.08    0.11     0.13 mi        -      m3
172.19.0.3           48          67   0    0.08    0.11     0.13 mdi       -      es2
```

4. _cat/shards показує перерозподіл primary/replicaміж es0, es2, d5, d6 (наприклад, документний шард my-lab-students shard 2: primary уже на d5, replica на es2), типова поведінка після падіння вузлу з даними.

```
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/shards?v"
index                shard prirep state   docs  store ip         node
my-lab-students      3     r      STARTED    0   261b 172.19.0.8 d6
my-lab-students      3     p      STARTED    0   261b 172.19.0.3 es2
my-lab-students      1     p      STARTED    0   261b 172.19.0.8 d6
my-lab-students      1     r      STARTED    0   261b 172.19.0.9 d5
my-lab-students      2     r      STARTED    2   10kb 172.19.0.3 es2
my-lab-students      2     p      STARTED    2   10kb 172.19.0.9 d5
my-lab-students      4     r      STARTED    0   261b 172.19.0.9 d5
my-lab-students      4     p      STARTED    0   261b 172.19.0.2 es0
my-lab-students      0     p      STARTED    0   259b 172.19.0.8 d6
my-lab-students      0     r      STARTED    0   259b 172.19.0.2 es0
.kibana_task_manager 0     p      STARTED    2 12.5kb 172.19.0.8 d6
.kibana_task_manager 0     r      STARTED    2  6.8kb 172.19.0.3 es2
.kibana_1            0     r      STARTED    2 10.4kb 172.19.0.3 es2
.kibana_1            0     p      STARTED    2 10.4kb 172.19.0.2 es0
task3-cluster-index  3     p      STARTED    0   261b 172.19.0.3 es2
task3-cluster-index  3     r      STARTED    0   261b 172.19.0.2 es0
task3-cluster-index  1     p      STARTED    2  8.4kb 172.19.0.8 d6
task3-cluster-index  1     r      STARTED    2  8.4kb 172.19.0.9 d5
task3-cluster-index  2     p      STARTED    2  8.3kb 172.19.0.3 es2
task3-cluster-index  2     r      STARTED    2  8.3kb 172.19.0.2 es0
task3-cluster-index  4     r      STARTED    0   261b 172.19.0.9 d5
task3-cluster-index  4     p      STARTED    0   261b 172.19.0.2 es0
task3-cluster-index  0     r      STARTED    1  4.2kb 172.19.0.8 d6
task3-cluster-index  0     p      STARTED    1  4.2kb 172.19.0.9 d5
```

5. docker compose up -d es1 - контейнер es1 знов підняли («up» у сенсі повернення вузола в топологію).

```
[+] up 2/2UTALTB07GO:~/elasticsearch-lab$ docker compose up -d es1
 ✔ Container es0 Running
 ✔ Container es1 Started                                                    user@WIN-
user@WIN-6UTALTB07GO:~/elasticsearch-lab$ curl -s "http://127.0.0.1:9200/_cat/nodes?v"
ip         heap.percent ram.percent cpu load_1m load_5m load_15m node.role master name
172.19.0.5           50          76   0    0.07    0.13     0.13 mdi       -      es1
172.19.0.6           63          76   1    0.07    0.13     0.13 mi        *      m4
172.19.0.2           55          76   1    0.07    0.13     0.13 mdi       -      es0
172.19.0.8           61          76   1    0.07    0.13     0.13 di        -      d6
172.19.0.9           64          76   1    0.07    0.13     0.13 di        -      d5
172.19.0.3           60          76   1    0.07    0.13     0.13 mdi       -      es2
172.19.0.4           44          76   1    0.07    0.13     0.13 mi        -      m3
user@WIN-6UTALTB07GO:~/elasticsearch-lab$
```

Після зупинки контейнеру сервісу es1, який виконував роль активного майстербузола, перевірено, що кластер зохранив працездатність із шістьма активним вузлам і статусом green, було обрано нового майстербузола (вузол m4 за результатом _cat/nodes). Таблицею _cat/shards продемонстровано зміну розміщення primary та replica-копій для індексів my-lab-students, task3-cluster-index тощо після виключення вузлу зберігання даних. Далі сервіс es1 знов піднято командою docker compose up -d es1; для опису завершального стану топології кластеру після повторного приєднання вузла повторено запити до _cluster/health, _cat/nodes та _cat/shields.

Висновок.

При виконанні лабораторної роботи, було розгорнуто кластер Elasticsearch у середовищі Ubuntu за допомогою Docker Compose, що дозволило відтворити типовий сценарій розподільного зберігання та індексування документів. Практично підтверджено ключові поняття кластера, ролей вузлів, первинних і реплікованих шардів, а також зміст понять стану узгодженості через запити до службового API. Такий підхід показує, що інструменти моніторингу кластера дають узгоджену картину доступності індексів і розподілу даних між вузлами.

На етапі ознайомчої частини сформовано власний індекс з тестовими даними, відмінними від навчальних прикладів методичних матеріалів, виконано індексацію з автоматичною генерацією ідентифікатора та із заданим ідентифікатором і відпрацьовано різні режими пошуку, зокрема через параметр запиту в адресі ресурсу та через Query DSL. Це продемонструвало, як одна модель зберігання підтримує і повнотекстовий аналіз, і структурні умови, зокрема діапазонні та шаблонні.

Окремо досліджено узгодженість даних між копіями шардів через аналітичні представлення расподілу та через читання одного документа напряму з вузлу, який відповідає репліці відповідного шарду. Оновлення існуючих записів за допомогою PUT показує механіку версіонування документа і поширення змін між первинним носієм і репліками в стабільному кластері.

Розширення топології за рахунок вузлів із відокремленням ролей майстербузлиних процесів і виключним зберіганням даних дозволило побачити вплив параметрів виявлення вузлів і кворуму на здатність кластеру утримувати керування метаданими при більш складній конфігурації. На набільшому набір узлів створено окремий індекс і проіндексовано фіксовану кількість документів із подальшою перевіркою вибірок за окремими полями.
Експеримент із зупинкою поточного майстербузола підтвердив автоматичне перевиборення керівного вузла та перерозклад або перерозподіл призначень шардів після виключення вузлу із даними, з подальшим поверненням вузла у склад кластера.

Додаток А

```
version: '2.4'

services :
   es0 :
   image : docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
   container_name : es0
   environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = es0
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2"
- "discovery.zen.minimum_master_nodes=2"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
ulimits:
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol0 : / usr / share / elasticsearch / data
ports :
-"9200:9200"
networks :
   -esnet

   es1 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : es1
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = es1
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2"
- "discovery.zen.minimum_master_nodes=2"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol1 : / usr / share / elasticsearch / data
networks :
-esnet

es2 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : es2
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
```

```
- node.name = es2
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2"
- "discovery.zen.minimum_master_nodes=2"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol2 : / usr / share / elasticsearch / data
networks :
-esnet

kibana :
image: docker.elastic.co / kibana / kibana : 6.8.6
container_name : kibana
environment :
-ELASTICSEARCH_URL = http ://es0:9200
   ports :
   -"5601:5601"
   depends_on :
   -es0
   networks :
-esnet

networks :
esnet: {}

volumes:
esvol0:
driver: local
esvol1 :
driver: local
esvol2 :
driver: local
```

Додаток В

```
services:
es0:
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : es0
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = es0
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
ulimits:
```

```
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol0 : / usr / share / elasticsearch / data
ports :
-"9200:9200"
networks :
   -esnet

   es1 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : es1
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = es1
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol1 : / usr / share / elasticsearch / data
networks :
-esnet

es2 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : es2
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = es2
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvol2 : / usr / share / elasticsearch / data
networks :
-esnet
```

```
m3 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : m3
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = m3
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "node.master=true"
- "node.data=false"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvolm3 : / usr / share / elasticsearch / data
networks :
-esnet

m4 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : m4
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = m4
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "node.master=true"
- "node.data=false"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvolm4 : / usr / share / elasticsearch / data
networks :
-esnet

d5 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : d5
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = d5
```

```
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "node.master=false"
- "node.data=true"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvold5 : / usr / share / elasticsearch / data
networks :
-esnet

d6 :
image: docker.elastic.co / elasticsearch / elasticsearch : 6.8.6
container_name : d6
environment :
-ES_SKIP_SET_KERNEL_PARAMETERS = true
- cluster.name = docker - cluster
- node.name = d6
- bootstrap.memory_lock = true
- "ES_JAVA_OPTS=-Xms256m -Xmx256m"
- "node.master=false"
- "node.data=true"
- "discovery.zen.ping.unicast.hosts=es0,es1,es2,m3,m4,d5,d6"
- "discovery.zen.minimum_master_nodes=3"
- "xpack.security.enabled=false"
- "xpack.watcher.enabled=false"
- "xpack.monitoring.enabled=false"
depends_on:
-es0
ulimits :
memlock:
soft: -1
hard : -1
mem_limit : 1g
volumes :
-esvold6 : / usr / share / elasticsearch / data
networks :
-esnet

kibana :
image: docker.elastic.co / kibana / kibana : 6.8.6
container_name : kibana
environment :
-ELASTICSEARCH_URL = http ://es0:9200
   ports :
   -"5601:5601"
   depends_on :
   -es0
   networks :
-esnet

networks :
esnet: {}
```

volumes:
esvol0:
driver: local
esvol1 :
driver: local
esvol2 :
driver: local
esvolm3 :
driver: local
esvolm4 :
driver: local
esvold5 :
driver: local
esvold6 :
driver: local